

Platform for Protecting Water Chlorination Systems from Cyber Attacks

Phase 1 / MVP Progress Report — A Process-Aware OT/ICS Anomaly-Detection and Monitoring System

Field	Value
Student Name(s)	Marko; Mahmoud; Kirolos; Keroles Yousef (team graduation project)
Student ID	Not recorded in the repository
Program	M.S. in Computer Science
Supervisor / Advisor	Dr. Zulkernine
Co-Advisor	None recorded
Date Submitted	September 12, 2026
Track / Domain	OT / ICS Security
Repository HEAD	e57a18d (branch docs/root-readme)

Student names are taken from the project proposal; student IDs are not recorded in the repository and are left unasserted rather than invented.

1. Abstract

Water-treatment plants are operational-technology (OT) environments in which cyber incidents can affect both digital systems and physical process behaviour. This report documents Phase 1 of a process-aware anomaly-detection and monitoring platform for water-chlorination ICS. The delivered MVP productionizes a previously selected and trained TranAD detector for the Secure Water Treatment (SWaT) testbed and integrates it into an end-to-end local monitoring pipeline. SWaT telemetry is replayed through a caller-owned rolling window of 30 samples and 45 process variables, scored by a stateless TranAD detector behind a thin FastAPI boundary, converted into deduplicated lifecycle-managed alerts, and exposed through monitoring APIs and an operator dashboard. The immutable model artifacts are reused without modification. Under the primary point-wise evaluation protocol at the shipped threshold, the system achieves precision 0.7822, recall 0.7821, F1 0.7822, and a 3.0117% false-positive rate; threshold-free ROC-AUC and PR-AUC are 0.9429 and 0.8516, respectively. The implementation is supported by 286 automated tests and an end-to-end normal-attack-recovery demonstration over a real HTTP boundary. The main contribution of Phase 1 is therefore not a new detector, but a reproducible and honestly evaluated engineering pipeline that turns a validated anomaly model into an operational monitoring MVP. Cloud deployment, containerization, CI/CD security gates, persistent alert storage, and further hardening remain future work.

2. Introduction / Background

Operational-technology (OT) and industrial control systems (ICS) run the physical processes behind critical infrastructure. In a water-treatment plant, programmable logic controllers (PLCs) read sensors and actuate pumps and valves to keep chlorine dosing within a safe range — too little leaves water unsafe to drink, too much makes it hazardous. OT/ICS environments are engineered primarily around process safety, reliability, and availability, while their security requirements and threat models differ from those of conventional IT systems. Modern connectivity, remote access, and integration with enterprise systems can increase the exposure of these environments, making security controls that account for both cyber and physical consequences essential [9], [10].

Process-aware detection matters because the most dangerous ICS attacks are not malformed packets but semantically valid commands with malicious physical intent: altering a dosing setpoint, spoofing a sensor so operators see normal while the plant drifts unsafe, injecting unauthorized commands, or disrupting the process. Each can unfold quietly without tripping a conventional network alarm, and the consequence is cyber-physical. Network- and perimeter-focused monitoring remains necessary, but it may be insufficient when an attacker uses legitimate control mechanisms to induce unsafe or anomalous physical behaviour. Process-aware detection complements these controls by evaluating whether observed multivariate telemetry remains consistent with learned plant behaviour.

This project takes a process-aware stance, modelling the plant's multivariate telemetry and flagging states the model cannot reconstruct. Concretely, this Phase 1 report delivers a working local MVP that wraps a transformer-based TranAD anomaly detector — already selected and trained on the SWaT testbed during an earlier research phase — inside an engineered, testable, operator-facing monitoring system, and evaluates it under a deployment-oriented protocol rather than a headline figure.

3. Problem Statement

How can anomalous, cyber-induced changes in multivariate water-treatment process telemetry be detected and surfaced to an operator through a usable monitoring system?

Sub-problems addressed this phase: (i) reproduce the preprocessing and scoring contract deterministically, with regression tests and golden-vector parity checks protecting the load-bearing transformations, so the pipeline never silently changes the quantity the threshold was calibrated against; (ii) turn a detector that scores one fixed-size window into a streaming system that buffers live telemetry correctly; (iii) convert per-window decisions into a small number of deduplicated incidents rather than one alert per window; and (iv) report detection quality under a protocol that reflects deployment. The problem is bounded to testbed data and defensive monitoring; it does not include controlling a real plant.

4. Research Objectives

The objectives below are paraphrased from the project proposal and the project's research question ("can a PLC-aware anomaly-detection system reliably identify malicious setpoint changes or abnormal process behaviour using process data?"). Each is annotated with its current status.

#	Objective (from proposal)	Status	Evidence in current work
O1	Analyze the SWaT dataset (normal behaviour and labelled attack scenarios).	ACHIEVED	Verified provenance record; clean Attack_v0 reconstruction (449,919 rows, 12.14% row-level attacks, 35 label segments).
O2	Integrate and evaluate the selected process-aware anomaly-detection model under a documented deployment-oriented protocol.	ACHIEVED	TranAD selected/trained in research phase; integrated end-to-end and re-measured point-wise (§7).
O3	Deliver a monitoring platform with a real-time operator dashboard.	ACHIEVED (local)	FastAPI monitoring API + zero-build dashboard over in-memory alert state.
O4	Demonstrate detection live on SWaT attack scenarios (low latency, low false positives).	PARTIALLY ACHIEVED	End-to-end normal→attack→recovery demo over real HTTP; FPR ~3% documented, not yet reduced.
O5	Treat detection as a secure software product: cloud, containers, CI/CD with security gates.	FUTURE	Explicitly out of scope for Phase 1; tracked in roadmap (§8.2, §9).
O6	Produce technical documentation and a final live attack demonstration.	PARTIALLY ACHIEVED	Provenance, architecture, and module docs exist; final demo/presentation pending.

5. Related Work / Literature Review

ICS / water-treatment security. Authoritative guidance frames OT security around the safety, reliability, and availability of physical processes and the distinct threat models this implies [9], with sector-specific guidance for water and wastewater systems [10]. The SWaT testbed was built by iTrust (SUTD) to support research on securing water treatment, and its public dataset records eleven days of operation with labelled attacks [1], [2]. The companion WADI water-distribution testbed extends this to distribution networks [3]. Such testbeds underpin most recent ICS-anomaly-detection work because real plant attack data is otherwise unavailable.

Multivariate, reconstruction-based detection. Process-aware detection is typically framed as unsupervised multivariate time-series anomaly detection: learn normal dynamics, then flag states with high reconstruction or prediction error. USAD couples adversarially-trained autoencoders [4]; GDN learns sensor relationships with graph neural networks [5]. TranAD [6] — the model productionized here — uses a transformer with two decoders and a focus-score mechanism, reconstructing the final timestep of each window and scoring the blended reconstruction error; it is reported as strong on SWaT/WADI benchmarks while remaining lightweight enough for CPU inference.

Rigorous evaluation. A recurring methodological hazard is the point-adjustment protocol, which marks a whole attack segment detected if any single point in it is flagged. Kim et al. [7] show this systematically overstates

performance — even a random detector scores highly — so point-adjusted F1 must not be quoted as a deployment figure. This directly shapes the evaluation in §7, where point-wise metrics are treated as primary.

ML systems / productionization. Deploying detection as software draws on ML-systems and DevSecOps practice — immutable versioned artifacts, reproducible preprocessing, thin service boundaries, and (later) security gates in CI/CD [8]; this work applies the first three now and defers pipeline hardening.

6. Proposed Approach / Methodology

The MVP is a layered pipeline with deliberately clean boundaries. The governing architectural principle is a strict separation of concerns: the client owns buffering, the detector is stateless, the API is a thin boundary, and exactly one stage (the alert engine) is stateful. Figure 1 shows the current (not generic) data flow.

SWaT telemetry → **SWaTReplay** → **RollingWindow** → **30 × 45 window** → **FastAPI /score** → **TranADDetector** → **Anomaly score + 45-way attribution** → **AlertEngine** → **Monitoring API** → **Operator Dashboard**

SWaT telemetry	licensed clean Attack_v0 CSVs (not committed)
SWaTReplay	ordered 1 Hz stream, subsample 5 → 5 s cadence
RollingWindow	owns the 30-sample ring buffer, stride 1
30 × 45 window	raw telemetry, feature-name order
FastAPI /score	thin, stateless boundary: validate → delegate
TranADDetector	stateless: preprocess → reconstruct → score
Anomaly score + 45-way attribution	per-feature reconstruction error
AlertEngine	stateful, in-memory: dedup, OPEN → CLOSED
Monitoring API	/status, /alerts, /alerts/active
Operator Dashboard	zero-build HTML/CSS/JS, polling

Figure 1. Current MVP detection pipeline (Phase 5A). Buffering stays in RollingWindow; scoring stays behind the API; incident state lives only in AlertEngine (in-memory).

Key architectural decisions (elaborating Figure 1).

- **RollingWindow owns the buffer** — a deque of maxlen 30 gives automatic stride-1 eviction; it validates feature count and finiteness, copies inputs, and emits a self-describing (30×45) window, importing neither torch nor FastAPI.
- **FastAPI is a thin, stateless boundary** — it validates shape/feature identity and order, delegates, and shapes the response; it does NOT buffer, does NOT duplicate ML logic, and holds no per-request or streaming state (dependency direction strictly backend → ml.src).
- **TranADDetector is stateless** — one (30×45) window in, one DetectionResult out, keeping it trivially testable and horizontally scalable.
- **AlertEngine is the one stateful layer** — a small state machine turns many consecutive anomalous windows into ONE alert (OPEN → dedup → CLOSE), tracking the peak window; severity is an explicitly-labelled heuristic (score/threshold ratio), not an ML output. Alert state is in-memory only.

The TranAD model.

TranAD is the selected and trained MVP detector; model selection is complete and is not revisited in this phase. The model consumes a 30-timestep window of 45 features. Because subsampling retains every fifth 1 Hz sample, the effective cadence is 5 s, so a 30-sample window represents approximately 150 s of temporal context and only becomes scoreable once 30 samples have accumulated (a single reading is not scoreable); the exact elapsed buffer-fill time depends on the timestamp convention used for the first sample. 30 samples are required because the transformer's positional encoding and window contract are fixed at training time; 45 features remain after six near-constant columns were dropped during training ($\text{std} \leq 1\text{e-}8$). A rolling window is therefore needed to assemble that context online from a one-sample-at-a-time feed.

Scoring reconstructs only the window's last timestep with two decoders; the anomaly score is the equal-weight blend $0.5 \cdot \text{MSE}(x_1, \text{elem}) + 0.5 \cdot \text{MSE}(x_2, \text{elem})$ averaged over the 45 features, compared strictly against the fixed threshold. Taking the squared error before the feature-dimension mean yields a 45-length per-feature error vector — which sensors drove the alert — at zero extra cost; this attribution is the concrete evidence for the process-aware claim and is carried through to the operator.

6.1 Data

Source and access. The dataset is the iTrust SWaT (Secure Water Treatment) testbed data, licensed from SUTD and held locally only (CSV files are excluded from version control). Three files are present: attack.csv (54,621 rows), normal.csv (1,387,098 rows), and merged.csv. merged.csv is excluded because it carries the identical data defect described below and is not a cleaner alternative.

Clean reconstruction. Rather than scoring the raw concatenation, the replay uses a reconstructed clean Attack_v0: attack.csv plus the genuine attack-period normal rows, merged by timestamp into the original normal → attack → normal sequence — 449,919 rows, zero missing values, all 45 features populated, at a 12.14% row-level attack ratio, with labels forming 35 contiguous attack segments. After windowing (length 30, stride 1), the evaluated last-timestep samples have an attack prevalence of approximately 12.15%. (The

proposal's "36 attacks" refers to attacks launched on the testbed; back-to-back attacks merge into 35 contiguous label runs.)

Preprocessing chain (reproduced deterministically from the research pipeline; regression tests and golden-vector parity checks protect the load-bearing transformations): select and order the 45 features per feature_names.json → StandardScaler (per-feature, float64) → cast to float32 → C-order (timestep-major) flatten to a 1350-dimensional vector → MinMaxScaler with clip=False → reshape back to (30×45). Three properties are load-bearing and each fails silently if broken, so each has a regression test: C-order flattening (index = t·45 + f), clip=False (attack windows scale outside [0,1], which is the detection signal), and the full 1350-column MinMax artifact (kept to match the stored artifact exactly). Subsample = 5 gives the 5 s cadence and ~150 s context. Unlike the research notebook, the production preprocessor rejects non-finite input instead of forward-filling it. Missing or non-finite telemetry is therefore rejected rather than silently imputed; dedicated detection of finite but stuck/frozen channels remains future work. The immutable ML artifacts themselves are byte-identical and SHA-256 verified.

Provenance findings and their engineering impact.

Auditing the data surfaced four findings that shaped design and evaluation. (1) Duplicated normal block: ~991,800 rows of normal.csv are the pre-attack run present twice, deflating the raw attack ratio to 3.79%. (2) Header-spelling mismatch: seven columns carry a leading space in the attack file and are empty in the duplicated block; the notebook's forward-fill then froze six model features to constants. (3) Consequently the shipped threshold (99th percentile of normal-validation scores) was calibrated on frozen-feature data, so its nominal 1% false-positive rate does not hold on complete telemetry. (4) On the clean slice the only genuinely constant feature is P301, confirming the training-time frozen features were the data gap, not physical constants. These findings led to using the clean Attack_v0 for replay/evaluation, surfacing the threshold caveat rather than silently correcting it, rejecting (not imputing) non-finite input, and reporting point-wise metrics at the true base rate.

6.2 Implementation Plan

Stack (verified versions). Python 3.10.12; PyTorch 2.9.1 (CPU) for TranAD inference; scikit-learn 1.7.2 (the two scalers, version-pinned by the joblib artifacts); NumPy 2.2.6; joblib 1.5.3 (artifact loading); FastAPI 0.141.1 with Uvicorn 0.52.4 (HTTP boundary); httpx 0.28.1 (client); vanilla HTML/CSS/JavaScript (zero-build dashboard); and pytest 8.4.2 (286 tests). Inference runs on CPU — no GPU is required for the MVP.

Module boundaries.

- **ml/** — production TranAD inference (model, preprocessing, scoring, detector, metrics) plus immutable artifacts and a verified config; research notebooks are kept separate under research/.
- **simulator/** — SWaTReplay, RollingWindow, and the end-to-end pipeline glue; imports neither torch nor FastAPI.
- **backend/** — the thin FastAPI app and request/response schemas.
- **alerting/** — the stateful alert engine and the Alert value object; duck-types on the detector's output, importing none of torch/FastAPI/ml.src/simulator.
- **frontend/** — the single-file operator dashboard.
- **tests/** — unit, ML-inference (golden-vector), API, integration, and end-to-end suites. docs/ holds proposal, provenance, and architecture material.

Implementation sequence (actual).

Phase 0 (provenance/environment) → Phase 1 (TranAD inference library + golden vectors) → Phase 1.5 (threshold-honest metrics) → Phase 2A (SWaT replay + rolling window) → Phase 2B (FastAPI) → Phase 3 (end-to-end integration) → Phase 4 (alert engine) → Phase 5A (monitoring API + dashboard). Notably, Phase 2A was built before Phase 2B on purpose: the rolling-window ownership question had to be resolved first so the API could be designed as a thin, stateless boundary with the 30-sample buffer owned entirely by the caller, rather than smuggling buffering state into the service.

7. Evaluation Plan and Preliminary Results

Protocol. Detection quality is reported point-wise — each window scored against ground truth at its last timestep, which is the only sample TranAD reconstructs. Point-wise evaluation is adopted as the primary deployment-oriented protocol because the widely used point-adjusted protocol marks an entire attack segment detected from a single flagged point and, per Kim et al. [7], systematically overstates performance. Threshold-free metrics (ROC-AUC, PR-AUC) are also reported to characterize the model independently of the shipped threshold.

Primary deployment-oriented metrics (point-wise, clean Attack_v0, at the shipped threshold 9.269035945180804e-05): Precision = 0.7822, Recall = 0.7821, F1 = 0.7822, and false-positive rate = 3.0117%. Threshold-free: ROC-AUC = 0.9429 and PR-AUC = 0.8516 (at the true ~12.15% windowed attack prevalence). These point-wise figures at the shipped threshold are the numbers to quote for deployment.

Why not the higher numbers. An F1 ≈ 0.9999 in earlier research material comes from a threshold searched on the test set; it is invalid as a deployment headline and must not be reported (the notebook warns that column "can read ~1.0 even for a broken model"). Point-adjusted results are reported only as secondary diagnostic comparisons: 0.7934 on the defective research concatenation, 0.9012 under any-in-window adjustment on clean Attack_v0, and 0.8815 under last-timestep point adjustment. These figures are not used as the primary

deployment metric. The ladder below changes one variable per rung — dataset defect, labelling convention, then point adjustment — so the drop to the deployment figure is attributable, not merely smaller.

Rung	Dataset / labelling / protocol	Precision	Recall	F1	FPR
1	research concat (defective) / any-in-window / point-adjusted	0.6575	1.0000	0.7934	—
2	clean Attack_v0 / any-in-window / point-adjusted	0.8450	0.9655	0.9012	—
3	clean Attack_v0 / last-timestep / point-adjusted	0.8151	0.9597	0.8815	—
4	clean Attack_v0 / last-timestep / point-wise (deployment)	0.7822	0.7821	0.7822	3.0117%

Table 1. Evaluation ladder. Rung 4 is the primary deployment metric; rungs 1-3 are secondary point-adjusted diagnostics. Point adjustment converts false negatives to true positives without changing FP/TN, so FPR is reported only for the point-wise deployment rung (see docs/provenance/measured_detection_metrics.md).

System-level validation.

- **Automated tests** — the full suite is 286 tests passing (pytest 8.4.2), spanning unit, golden-vector ML-inference parity, backend API, monitoring API, Phase 3 end-to-end integration, and Phase 4 alert-engine cases.
- **Model validation** — golden-vector tests assert bit-level reconstruction parity; regression tests guard the silent-corruption properties (C-order flatten, clip=False).
- **End-to-end / HTTP** — the pipeline is verified on real SWaT replay, both in-process (ASGI) and over a real Uvicorn socket, exercising the normal → attack → recovery lifecycle through /score into the alert engine.
- **Manual smoke testing** — the operator dashboard renders live status and alerts against the running API.

Demo vs. metric. The end-to-end demo auto-selects the attack segment that reads most clearly at onset and can report 100% detection on that segment; this is a demonstration result for one chosen segment, not a full-dataset performance figure. The full-dataset, deployment-oriented numbers are those in the table above.

8. Expected Outcomes / Deliverables

8.1 Achieved outcomes / current MVP.

Delivered and tested: a TranAD inference engine (stateless detector over immutable artifacts); a reproducible, regression-tested preprocessing pipeline (golden-vector parity on the load-bearing transforms); SWaT telemetry replay of the clean Attack_v0 series; a caller-owned rolling 30-sample window; a thin, stateless FastAPI backend (/health, /score); a stateful in-memory alert engine (dedup + OPEN→CLOSED lifecycle); a monitoring API (/status, /alerts, /alerts/active) with a zero-build operator dashboard; an end-to-end demonstration over a real HTTP boundary (normal/attack/recovery); deployment-oriented evaluation (point-wise metrics + provenance); and 286 automated tests across unit, ML, API, integration, and end-to-end suites.

8.2 Remaining outcomes for final delivery (future work).

Not yet implemented: alert persistence/database (replacing in-memory state); stuck/frozen-channel detection at ingest; Docker containerization; a CI/CD pipeline with integrated DevSecOps security gates (SAST, dependency and container scanning); AWS deployment and Terraform infrastructure-as-code; Kubernetes orchestration (a stretch goal); authentication and alert notifications; observability (metrics/logging/tracing); final hardening and performance/latency testing; and an optional, approval-gated threshold recalibration on a clean split (sibling artifact, no retraining). All of these are future work and are not present in the current repository.

9. Timeline

Original proposal plan (proposal Slide 9 sequencing, expressed in the template's Phase/Task/Weeks form):

Phase	Task	Weeks
1	Literature review + SWaT dataset analysis	1-2
2	Data collection & normalization pipeline	3-4

Phase	Task	Weeks
3	Train & evaluate ML anomaly-detection model	5–7
4	Cloud infrastructure + monitoring dashboard (in parallel)	6–9
5	CI/CD pipeline + security-gate integration	9–11
6	Testing, validation, performance tuning	11–13
7	Final demonstration + technical documentation	13–14

Actual progress.

Proposal Phases 1–3 are complete: the dataset was analyzed and its provenance verified, the preprocessing pipeline reproduced, and the TranAD model integrated, evaluated, and honestly re-measured. Proposal Phase 4 is partially complete — the monitoring dashboard and API are delivered locally, but cloud infrastructure is not. The implementation evolved in finer-grained engineering phases (0, 1, 1.5, 2A, 2B, 3, 4, 5A; see §6.2), and the build order was deliberately adjusted so telemetry replay and the rolling window (2A) preceded the API (2B). Remaining future work corresponds to proposal Phases 5–7 (CI/CD security gates, full validation/performance tuning, final demo) plus cloud deployment and alert persistence — none of which are implemented yet.

10. Risks and Mitigation

Risk	Likelihood	Impact	Mitigation
SWaT data-quality / provenance defects (duplicated block, header mismatch, frozen features)	Occurred	High	Fully documented; clean Attack_v0 reconstruction used; findings version-controlled.
Threshold caveat → elevated false positives (~3% FPR; nominal 1% not held)	Occurred	Medium	Caveat surfaced in code/docs; honest point-wise metrics reported; recalibration deferred and approval-gated. Alert dedup reduces repeat notifications and severity ranks incidents, but neither changes the detector FPR.
150 s temporal-context requirement (buffer must fill before the first score)	Certain	Low-Med	Inherent to the model; a full 30-sample window is needed before scoring; real-time-factor knob for demos.
Stuck / plausibly-frozen sensor channel passes silently	Medium	Medium	Non-finite input rejected now; dedicated stuck-channel check planned at ingest.
Alert state lost on restart (in-memory only)	Certain	Medium	Documented limitation; persistence/database is prioritized future work.
Dataset access / licensing (not redistributable)	Low	Medium	CSVs kept local and git-ignored; obtained under iTrust license; path via env var.
Artifact reproducibility / environment drift	Low	High	Artifacts immutable with SHA-256 pins; scikit-learn/torch/numpy pinned; golden-vector parity test.

11. Resources Required

Development environment: a Linux workstation with a standard CPU — no GPU is required for TranAD inference or the current evaluation. Python environment (verified, pinned): Python 3.10.12 with PyTorch 2.9.1 (CPU), NumPy 2.2.6, scikit-learn 1.7.2, joblib 1.5.3, FastAPI 0.141.1, Uvicorn 0.52.4, httpx 0.28.1, and pytest 8.4.2; the dashboard is vanilla HTML/CSS/JS with no build toolchain. Dataset: the licensed iTrust SWaT CSVs,

held locally (not in version control) and provided via \$SWAT_DATASET_DIR. Tooling: Git/GitHub for source and documentation. Future cloud resources (needed only for the deferred cloud/CI-CD phases, not the current local MVP): an AWS account, a container registry, Terraform, and optionally Kubernetes. The project also requires ongoing team collaboration and periodic supervisor review.

12. References

- [1] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in Proc. Int. Conf. Critical Information Infrastructures Security (CRITIS), 2016, pp. 88–99.
- [2] A. P. Mathur and N. O. Tippenhauer, "SWaT: A water treatment testbed for research and training on ICS security," in Proc. Int. Workshop on Cyber-Physical Systems for Smart Water Networks (CySWater), 2016, pp. 31–36.
- [3] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, "WADI: A water distribution testbed for research in the design of secure cyber physical systems," in Proc. 3rd Int. Workshop on Cyber-Physical Systems for Smart Water Networks (CySWATER), 2017, pp. 25–28.
- [4] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD), 2020, pp. 3395–3404.
- [5] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in Proc. AAAI Conf. Artificial Intelligence, vol. 35, no. 5, 2021, pp. 4027–4035.
- [6] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," Proc. VLDB Endowment, vol. 15, no. 6, pp. 1201–1214, 2022.
- [7] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon, "Towards a rigorous evaluation of time-series anomaly detection," in Proc. AAAI Conf. Artificial Intelligence, vol. 36, no. 7, 2022, pp. 7194–7201.
- [8] D. Sculley et al., "Hidden technical debt in machine learning systems," in Advances in Neural Information Processing Systems (NeurIPS), vol. 28, 2015, pp. 2503–2511.
- [9] K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, O. Saravia, A. Sherule, and M. Thompson, "Guide to Operational Technology (OT) Security," NIST Special Publication 800-82 Rev. 3, National Institute of Standards and Technology, Gaithersburg, MD, USA, Sep. 2023, doi: 10.6028/NIST.SP.800-82r3.
- [10] Cybersecurity and Infrastructure Security Agency (CISA), Federal Bureau of Investigation (FBI), and U.S. Environmental Protection Agency (EPA), Incident Response Guide: Water and Wastewater Sector, Jan. 2024.